CONTOUR TRAINING

Java & Backend MCQ Practice

54 Questions • Sections A–G • OOP · Java · HTTP · SQL · DSA · Git

SECTION A — OOP CONCEPTS MCQs

Q1. What is encapsulation in OOP?

A) The ability to inherit from multiple classes

B) Hiding internal data and exposing only what is necessary via methods ✓

C) Calling the same method in different ways

D) Breaking code into smaller functions

Answer: B) Hiding internal data and exposing only what is necessary via methods

Encapsulation = bundling data (fields) + methods, hiding internals with private. Access via getters/setters.

Q2. What keyword is used to inherit from a class in Java?

A) implements

B) inherits

C) extends ✓

D) super

Answer: C) extends

extends = class inheritance. implements = interface. super = call parent constructor/method.

Q3. What is abstraction?

A) Having multiple constructors

B) Hiding implementation details, showing only essential features ✓

C) Making all fields public

D) Creating objects from classes

Answer: B) Hiding implementation details, showing only essential features

Abstraction = show WHAT, hide HOW. Achieved via abstract classes and interfaces in Java.

Q4. Which OOP concept allows a child class to use methods of a parent class?

A) Polymorphism

B) Abstraction

C) Inheritance ✓

D) Encapsulation

Answer: C) Inheritance

Inheritance = child class reuses parent class code. extends keyword. Promotes code reuse.

Q5. What is an interface in Java?

A) A class with only static methods

B) A blueprint with abstract methods that classes must implement ✓

C) A class that cannot be instantiated

D) A class with private constructor

Answer: B) A blueprint with abstract methods that classes must implement

Interface = contract. All methods are abstract (before Java 8). Class must implement ALL methods.

Q6. What does the 'static' keyword mean for a method?

A) The method can only be called once

B) The variable/method belongs to the class, not to any instance ✓

C) The method is inherited by subclasses

D) The method runs automatically on startup

Answer: B) The variable/method belongs to the class, not to any instance

Static = shared across all instances. Called via ClassName.method(). No 'this' reference needed.

Q7. Which principle states a class should have only one reason to change?

A) Open/Closed Principle

B) Dependency Inversion

C) Single Responsibility Principle ✓

D) Interface Segregation

Answer: C) Single Responsibility Principle

SRP: one class = one job. Part of SOLID principles. Makes code easier to maintain and test.

Q8. What is polymorphism?

A) A class having multiple constructors

B) Same method name behaving differently based on context ✓

C) A class implementing multiple interfaces

D) Hiding class fields with private access

Answer: B) Same method name behaving differently based on context

Polymorphism = many forms. Compile-time: overloading. Runtime: overriding. Animal.speak() differs for Dog and Cat.

Q9. What is a constructor?

A) A method that returns the class type

B) A static method that initializes the class

C) A special method with same name as class, no return type, called on object creation ✓

D) A method that destroys an object

Answer: C) A special method with same name as class, no return type, called on object creation

Constructor initializes fields when object is created. If not defined, Java provides a default one.

Q10. Which access modifier makes a field accessible only within the same class?

A) public
C) default
B) protected
D) private ✓

Answer: D) private
private = same class only. protected = same package + subclass. public = anywhere. default = same package.

SECTION B — JAVA OUTPUT PREDICTION MCQs

Q11. What is the output of the following code?

```
int x = 5;
System.out.println(x++);
System.out.println(x);
```

A) 5 then 5
C) 6 then 6
B) 5 then 6 ✓
D) 6 then 7

Answer: B) 5 then 6
x++ is post-increment: print current value (5) THEN increment. Next line x is now 6.

Q12. What does this code print?

```
int x = 5;
System.out.println(++x);
System.out.println(x);
```

A) 5 then 6
C) 6 then 7
B) 6 then 6 ✓
D) 5 then 5

Answer: B) 6 then 6
++x is pre-increment: increment FIRST (5->6) then print. x stays 6 on next line.

Q13. What is the output?

```
String a = "hello";
String b = "hello";
String c = new String("hello");
System.out.println(a == b);
System.out.println(a == c);
System.out.println(a.equals(c));
```

A) true, true, true
C) false, false, true
B) true, false, true ✓
D) true, true, false

Answer: B) true, false, true
a and b point to same string pool. c is a new heap object. == checks reference. .equals() checks value.

Q14. What does this print?

```
class Animal {
  void speak() { System.out.println("Animal"); }
}
class Dog extends Animal {
  void speak() { System.out.println("Dog"); }
}
Animal a = new Dog();
a.speak();
```

A) Animal
C) Compilation error
B) Dog ✓
D) Animal then Dog

Answer: B) Dog

Runtime polymorphism. The actual object is Dog, so Dog.speak() is called. This is method overriding.

Q15. What is the output of this try-catch?

```
try {
  System.out.println("A");
  int x = 10/0;
  System.out.println("B");
} catch(ArithmeticException e) {
  System.out.println("C");
} finally {
  System.out.println("D");
}
```

A) A B C D
C) A C
B) A C D ✓
D) A B D

Answer: B) A C D

A prints. Division by zero throws exception. B is skipped. Catch prints C. Finally ALWAYS runs -> D.

Q16. What does this print?

```
class Counter {
  static int count = 0;
  Counter() { count++; }
}
new Counter();
new Counter();
new Counter();
System.out.println(Counter.count);
```

A) 0
C) 3 ✓
B) 1
D) Compilation error

Answer: C) 3

Static variable is shared across all instances. Each constructor call increments the same count.

Q17. What is the output?

```
int[] arr = {10, 20, 30, 40, 50};
System.out.println(arr.length);
System.out.println(arr[2]);
```

A) 5 then 20
C) 5 then 30 ✓
B) 4 then 30
D) 5 then 40

Answer: C) 5 then 30
arr.length is 5 (property, no brackets). arr[2] is index 2 = 30 (0-indexed: 10,20,30,40,50).

Q18. What prints?

```
String s = "Hello World";
System.out.println(s.length());
System.out.println(s.toUpperCase());
System.out.println(s.substring(6));
```

A) 11, HELLO WORLD, World ✓
C) 11, HELLO WORLD, orld
B) 10, hello world, World
D) 12, HELLO WORLD, World

Answer: A) 11, HELLO WORLD, World
length()=11 (space counts). toUpperCase() = HELLO WORLD. substring(6) = World (index 6 to end).

Q19. What is the result?

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);
list.remove(1);
System.out.println(list);
```

A) [10, 30] ✓
C) [10, 20]
B) [20, 30]
D) [10, 20, 30]

Answer: A) [10, 30]
remove(1) removes element at INDEX 1, which is 20. Remaining: [10, 30].

Q20. What does this print?

```
boolean x = true;
boolean y = false;
System.out.println(x && y);
System.out.println(x || y);
System.out.println(!x);
```

A) true, true, false
C) true, false, true
B) false, true, false ✓
D) false, false, true

Answer: B) false, true, false
true && false = false. true || false = true. !true = false.

SECTION C — FILL IN THE BLANK

Q21. Complete the class to implement encapsulation — fill in the blanks:

```
public class Person {
  _______ String name;   // blank 1
  _______ int age;       // blank 2

  public _______ getName() {  // blank 3
    return name;
  }

  public void setName(_______ n) {  // blank 4
    this.name = n;
  }
}
```

Answer: blank1=private, blank2=private, blank3=String getName(), blank4=String n

Encapsulation: private fields, public getter/setter. Return type must match field type.

Q22. Complete the inheritance — fill in the blank:

```
class Animal {
  void speak() {
    System.out.println("Animal speaks");
  }
}

class Dog _______ Animal {  // blank 1
  _______ void speak() {    // blank 2
    System.out.println("Dog barks");
  }
}
```

Answer: blank1=extends, blank2=@Override (optional but good practice)

extends keyword for inheritance. @Override annotation confirms you're overriding — not required but best practice.

Q23. Complete the interface implementation:

```
interface Drawable {
  void draw();  // abstract method
}

class Circle _______ Drawable {  // blank 1
  public void _______ {          // blank 2
    System.out.println("Drawing circle");
  }
}
```

Answer: blank1=implements, blank2=draw()

implements keyword for interfaces. Class MUST implement all interface methods or be declared abstract.

Q24. Fill in to make this a Singleton pattern:

```
public class Database {
  private _______ Database instance;  // blank 1

  private Database() {}  // private constructor

  public _______ Database getInstance() {  // blank 2
    if (instance == null) {
      instance = new Database();
    }
    return instance;
  }
}
```

Answer: blank1=static, blank2=static

Singleton: private static instance, private constructor, public static getInstance(). Only one object ever created.

Q25. Complete the ArrayList operations:

```
ArrayList<String> names = new ArrayList<>();
names._______("Alice");    // blank 1 - add element
names._______("Bob");      // blank 2 - add element
System.out.println(names._______()); // blank 3 - get size
String first = names._______(0);    // blank 4 - get first element
names._______(0);                   // blank 5 - remove first
```

Answer: blank1=add, blank2=add, blank3=size(), blank4=get, blank5=remove

ArrayList methods: add(), get(index), size(), remove(index), contains(), isEmpty()

Q26. Complete the try-catch-finally:

```
try {
  int result = 10 / _______;  // blank 1 - will throw exception
  System.out.println(result);
} catch (_______ e) {          // blank 2 - exception type
  System.out.println("Cannot divide by zero");
} _______ {                    // blank 3 - always runs
  System.out.println("Done");
}
```

Answer: blank1=0, blank2=ArithmeticException, blank3=finally

Division by zero throws ArithmeticException. finally block always executes regardless of exception.

Q27. Complete the for-each loop:

```
int[] numbers = {1, 2, 3, 4, 5};
int sum = 0;

for (_______ num : numbers) {  // blank 1 - type
  sum _______ num;             // blank 2 - accumulate
}
System.out.println(sum);  // prints 15
```

Answer: blank1=int, blank2=+= (sum += num)

for-each: for(Type variable : collection). sum += num is shorthand for sum = sum + num.

Q28. Complete the HashMap operations:

```
HashMap<String, Integer> scores = new HashMap<>();
scores._______("Alice", 95);  // blank 1 - insert
scores._______("Bob", 87);    // blank 2 - insert

int aliceScore = scores._______("Alice");   // blank 3 - retrieve
boolean hasKey = scores._______("Charlie"); // blank 4 - check key

for (String key : scores._______()) {  // blank 5 - iterate keys
  System.out.println(key + ": " + scores.get(key));
}
```

Answer: blank1=put, blank2=put, blank3=get, blank4=containsKey, blank5=keySet()

HashMap: put(key,value), get(key), containsKey(key), keySet() for iteration.

SECTION D — HTTP, REST & CACHING MCQs

Q29. Which HTTP method is used to CREATE a new resource?

A) GET

B) PUT

C) POST ✓

D) PATCH

Answer: C) POST

POST = create. GET = read. PUT = replace all. PATCH = partial update. DELETE = remove.

Q30. What HTTP status code means 'resource not found'?

A) 400

B) 401

C) 403

D) 404 ✓

Answer: D) 404

404 = Not Found. 400 = Bad Request. 401 = Unauthorized. 403 = Forbidden.

Q31. What is the difference between 401 and 403?

A) 401 = server error, 403 = client error

B) 401 = not logged in (unknown user), 403 = logged in but no permission ✓

C) They are the same

D) 401 = rate limited, 403 = not found

Answer: B) 401 = not logged in (unknown user), 403 = logged in but no permission

401: WHO are you? Not authenticated. 403: I know who you are, but you can't do this.

Q32. What does 'idempotent' mean in HTTP?

A) The request has no body

B) The request can only be made once

C) Calling the same request multiple times gives the same result ✓

D) The request is encrypted

Answer: C) Calling the same request multiple times gives the same result

GET, PUT, DELETE are idempotent. POST is not — calling POST twice creates two resources.

Q33. What is caching used for?

A) Encrypting user data

B) Storing frequently accessed data in fast memory to reduce DB load ✓

C) Sending data between microservices

D) Versioning API endpoints

Answer: B) Storing frequently accessed data in fast memory to reduce DB load

Cache = fast memory store. Less DB queries = faster response, lower cost, handles more traffic.

Q34. What does TTL stand for in caching?

A) Total Transfer Limit

B) Time To Load

C) Time To Live ✓

D) Token Transfer Layer

Answer: C) Time To Live

TTL = how long cached data stays before expiring. After TTL, next request goes to DB and refreshes cache.

Q35. What is Authentication vs Authorization?

A) Authentication = what you can do, Authorization = who you are

B) They are the same thing

C) Authentication = who you are (identity), Authorization = what you can do (permissions) ✓

D) Authentication happens after Authorization

Answer: C) Authentication = who you are (identity), Authorization = what you can do (permissions)

Auth-n = identity verification (login). Auth-z = permission checking (role-based access).

Q36. Which status code is returned when a new resource is successfully created?

A) 200

B) 201 ✓

C) 204

D) 202

Answer: B) 201

201 Created = resource was successfully created. 200 OK = general success. 204 = success, no body to return.

Q37. What is CORS?

A) A caching strategy

B) A browser security feature that blocks requests from different origins unless server allows ✓

C) A type of authentication

D) A database optimization technique

Answer: B) A browser security feature that blocks requests from different origins unless server allows

Cross-Origin Resource Sharing. Next.js on :3000 calling Django on :8000 = CORS issue. Fix: django-cors-headers.

Q38. What does REST stand for?

A) Remote Execution State Transfer

B) Representational State Transfer ✓

C) Resource Endpoint Service Technology

D) Remote Event Streaming Tool

Answer: B) Representational State Transfer

REST = architectural style for APIs. Uses HTTP methods. Stateless. Resources identified by URLs.

SECTION E — SQL MCQs

Q39. Which SQL clause filters rows BEFORE grouping?

A) HAVING

B) GROUP BY

C) WHERE ✓

D) ORDER BY

Answer: C) WHERE

WHERE filters individual rows before grouping. HAVING filters groups AFTER GROUP BY.

Q40. What does INNER JOIN return?

A) All rows from left table

B) All rows from both tables

C) Only rows with matching values in BOTH tables ✓

D) All rows from right table

Answer: C) Only rows with matching values in BOTH tables

INNER JOIN = intersection. LEFT JOIN = all left + matching right. FULL JOIN = all from both.

Q41. What does this query return?

```
SELECT dept, COUNT(*) as total
FROM employees
GROUP BY dept
HAVING COUNT(*) > 3;
```

A) All employees

B) Departments with more than 3 employees ✓

C) The first 3 departments

D) All departments with their count

Answer: B) Departments with more than 3 employees

GROUP BY groups rows by dept. HAVING filters groups — only depts with more than 3 employees shown.

Q42. What is a PRIMARY KEY?

A) A key that can have duplicate values

B) A key that references another table

C) A unique identifier for each row, cannot be null ✓

D) An index on a column

Answer: C) A unique identifier for each row, cannot be null

Primary key: unique + not null. One per table. Foreign key references another table's primary key.

Q43. What does this query return?

```
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC
LIMIT 3;
```

A) All employees sorted by salary

B) Top 3 highest paid employees earning more than 50000 ✓

C) Bottom 3 lowest paid employees

D) 3 random employees

Answer: B) Top 3 highest paid employees earning more than 50000

WHERE filters salary > 50000. ORDER BY salary DESC = highest first. LIMIT 3 = top 3 only.

Q44. What is the difference between DELETE and TRUNCATE?

A) They are the same

B) DELETE removes specific rows (with WHERE), TRUNCATE removes ALL rows faster, can't rollback ✓

C) TRUNCATE removes specific rows, DELETE removes all

D) DELETE is faster than TRUNCATE

Answer: B) DELETE removes specific rows (with WHERE), TRUNCATE removes ALL rows faster, can't rollback

DELETE: can filter with WHERE, slower, logged, rollback possible. TRUNCATE: all rows, faster, not logged.

SECTION F — DATA STRUCTURES MCQs

Q45. What is the time complexity of accessing an element in an ArrayList by index?

A) O(n)

B) O(log n)

C) O(1) ✓

D) O(n^2)

Answer: C) O(1)

ArrayList is backed by an array. Direct index access is O(1) — instant regardless of size.

Q46. What data structure uses LIFO (Last In First Out)?

A) Queue
B) Stack ✓
C) LinkedList
D) HashMap

Answer: B) Stack
Stack = LIFO. Last item pushed is first popped. Queue = FIFO. Like a stack of plates.

Q47. What is the average time complexity of HashMap.get()?

A) O(n)
B) O(log n)
C) O(n^2)
D) O(1) ✓

Answer: D) O(1)
HashMap uses hash function for direct lookup. O(1) average. O(n) worst case with many collisions.

Q48. Which sorting algorithm has O(n log n) time complexity in the average case?

A) Bubble sort
B) Selection sort
C) Merge sort ✓
D) Insertion sort

Answer: C) Merge sort
Merge sort always O(n log n). Quick sort O(n log n) average but O(n^2) worst. Bubble/Selection = O(n^2).

Q49. What is required for binary search to work?

A) Array must have even number of elements
B) Array must be sorted ✓
C) Array must have no duplicates
D) Array must be a LinkedList

Answer: B) Array must be sorted
Binary search halves search space each step. Only works on sorted data. Time: O(log n).

Q50. What is a linked list?

A) An array with fixed size
B) A tree structure with two children per node
C) A sequence of nodes where each node points to the next ✓
D) A hash-based data structure

Answer: C) A sequence of nodes where each node points to the next
LinkedList: nodes with data + next pointer. Fast insert/delete O(1), slow access O(n).

SECTION G — GIT MCQs

Q51. What does 'git clone' do?

A) Creates a new branch

B) Copies a remote repository to your local machine ✓

C) Saves your changes

D) Uploads changes to remote

Answer: B) Copies a remote repository to your local machine

git clone = download remote repo locally. Includes full history and all branches.

Q52. What is the correct order for pushing code to GitHub?

A) push -> add -> commit

B) commit -> add -> push

C) add -> commit -> push ✓

D) push -> commit -> add

Answer: C) add -> commit -> push

Stage changes (add), save snapshot (commit), upload to remote (push). Always in this order.

Q53. What does 'git stash' do?

A) Deletes all uncommitted changes permanently

B) Temporarily saves uncommitted changes so you can switch branches ✓

C) Creates a new branch

D) Merges two branches

Answer: B) Temporarily saves uncommitted changes so you can switch branches

git stash = save work in progress without committing. git stash pop = restore stashed changes.

Q54. What is a pull request?

A) Downloading code from remote repository

B) A request to merge your branch into another branch after team code review ✓

C) Requesting someone to pull your changes manually

D) A conflict resolution tool

Answer: B) A request to merge your branch into another branch after team code review

PR = code review gate before merging. Team reviews, comments, approves, then merges to main.

ANSWER KEY — QUICK REFERENCE

1-B	2-C	3-B	4-C	5-B
6-B	7-C	8-B	9-C	10-D
11-B	12-B	13-B	14-B	15-B
16-C	17-C	18-A	19-A	20-B
21-See	22-See	23-See	24-See	25-See
26-See	27-See	28-See	Fill	Blanks
29-C	30-D	31-B	32-C	33-B

34-C	35-C	36-B	37-B	38-B
39-C	40-C	41-B	42-C	43-B
44-B	45-C	46-B	47-D	48-C
49-B	50-C	51-B	52-C	53-B
54-B				